

FINAL INFORMATICA I Parte 1				Duración	Fecha	Hojas
				50 minutos	30/jul/2026	
Nombre y Apellido	Nº Legajo	Calificación parte 1		Docente Evaluador		
		número	letras	Nombre	Firma	

Numere las hojas entregadas, y complete el casillero *Hojas* con la cantidad.

Lea detenidamente el enunciado, su correcta interpretación forma parte de esta evaluación.

Requerido 3 puntos bien para avanzar a la parte 2.

1. Aritmética de bits

Analice el siguiente código e indique que imprime.

```
#define BIT(n) (1U << (n))

int main(void)
{
    uint8_t estado = 0xA4;

    estado |= BIT(0);
    estado &= ~BIT(7);

    printf("Estado: 0x%02X\n", estado);

    return 0;
}
```

2. Punteros

Analice el siguiente código e indique que imprime. En caso que la impresión sea indeterminada, indíquelo y justifique.

```
int main(void)
{
    int i;
    uint64_t datos[2] =
    {
        0x1122334455667788ULL,
        0x99AABBCCDDEEFF00ULL
    };

    uint64_t *p64 = datos;
    uint16_t *p16 = (uint16_t *)datos;
    uint8_t *p8 = (uint8_t *)datos;

    for (i=0 ; i<3 ; i++)
    {
        printf("%x\n", (uint8_t *)p64 - (uint8_t *)datos);
        printf("%x\n", (uint8_t *)p16 - (uint8_t *)datos);
        printf("%x\n", (uint8_t *)p8 - (uint8_t *)datos);

        p64++;
        p16++;
        p8++;
    }

    return 0;
}
```

3. Strings

La función **recortar** elimina los espacios existentes al final de una cadena de caracteres o strings, y en general funciona bien excepto en dos ocasiones a saber: cuando el string está vacío y cuando se recibe un string solo con espacios.

Explique por qué se dan los inconvenientes y corrija el código con mínimas intervenciones.

<pre>void recortar(char *text) { char *fin = text + strlen(text)-1; while (*fin == ' ') { *fin = '\0'; fin--; } }</pre>	<pre>int main(void) { char texto[20] = "RFID UHF "; recortar(texto); printf("[%s]\n", texto); return 0; }</pre>
---	--

4. Punteros

La función **buscarMayor** busca determinar el máximo dentro de un array elementos de **int**, con **cant** elementos. Sin embargo, al ejecutarse la función se produce un fallo de segmentación y el programa se cae.

Explique por qué se dan los inconvenientes y corrija el código con mínimas intervenciones.

<pre>int *buscarMayor(int *val, size_t cant) { int mayor = val[0]; for (size_t i = 1; i < cant; i++) { if (val[i] > mayor) mayor = *(val+i); } return &mayor; }</pre>	<pre>#define SZ(n) (sizeof (n)/sizeof (*n)) int main(void) { int dat[] = {8, 14, 3, 27, 9}; int *res = buscarMayor(dat, SZ (dat)); printf("%d\n", *res); return 0; }</pre>
--	---

5. Lista simplemente enlazada

La siguiente función busca determinar si el valor buscado está dentro de la lista, retornando 1 en caso de encontrarlo y 0 si eso no ocurre. Sin embargo, se dan ciertas situaciones en donde la aplicación se cae por causa de esta función, y en otras indica que el valor buscado no se encuentra cuando en realidad sí lo está.

Explique por qué se dan los inconvenientes y corrija el código con mínimas intervenciones.

<pre>typedef struct Nodo { int dato; struct Nodo *sig; } Nodo;</pre>	<pre>int Buscar(nodo_t *lista, int valor) { while(lista->sig != NULL) { if(lista->dato == valor) return 1; lista = lista->sig; } return 0; }</pre>
--	---